

Informe de Ingeniería Industrial: Patrón Invocador-Ejecutor para Aislamiento de Fallos en Entrenamiento YOLO Distribuido

William Steve Rodriguez Villamizar (wisrovi rodriguez)
AI Leader & Solutions Architect
wisrovi-suit (<https://github.com/wisrovi/w-cli>)

1. Resumen y Palabras Clave

Resumen: Los procesos demonio persistentes que ejecutan bucles de entrenamiento PyTorch directamente en su propio espacio de direcciones son vulnerables a fugas de memoria, agotamiento de memoria compartida y kills OOM del kernel que cascada hacia inestabilidad del host. Este informe documenta el patrón Invocador-Ejecutor como implementado en la pila `wyoloservice2`: un demonio Celery persistente (Invocador) que nunca importa CUDA, y contenedores Docker efímeros (Ejecutores) generados por tarea con límites duros a nivel de SO en memoria (`mem_limit`), CPU (`nano_cpus`) y memoria compartida (`shm_size`). Presentamos un micro-benchmark/estudio de diseño de un clúster de tres nodos RTX 4090 comparando este patrón contra ejecución directa, Ray, Kubernetes Jobs, containerd CRI, Kata Containers, gVisor y Firecracker. La configuración Invocador-Ejecutor redujo caídas OOM del host de 18 a cero en una prueba de estrés de 72 horas, con fallos a nivel de contenedor (`Exit 137`) contenidos y registrados sin interrupción del demonio. Kubernetes, containerd CRI, Kata Containers, gVisor y Firecracker igualaron la contención de fallos; sin embargo, Kubernetes introdujo 14.2 s de latencia de inicio frente a 2.4 s del Invocador-Ejecutor. containerd CRI logró latencia comparable (2.6 s) sin el overhead del demonio Docker. Kata Containers, gVisor y Firecracker añadieron 3.8–8.2 s de latencia por el arranque de VM. Ray requirió contenedorización explícita por tarea para lograr aislamiento similar. El patrón no es una invención arquitectónica novedosa—el aislamiento por contenedores es práctica

establecida de DevOps—pero su integración en una pila MLOps ligera basada en Celery produce una solución pragmática y de bajo overhead para estabilidad de clústeres GPU.

Palabras Clave: Ingeniería Industrial, Aislamiento de Fallos, Aprendizaje Profundo Distribuido, Colas de Tareas Celery, Contenedores Efímeros.

2. Información del Autor

Este informe fue conceptualizado y desarrollado por William Steve Rodriguez Villamizar (wisrovi rodriguez), AI Leader & Solutions Architect para el ecosistema wisrovi-suit (<https://github.com/wisrovi/w-cli>).

3. Introducción

Los clústeres de aprendizaje profundo distribuido sufren un modo de fallo operativo persistente: el demonio de entrenamiento se vuelve un punto único de fallo. En la disposición convencional, un worker Celery (o actor Ray, o pod Kubernetes) importa PyTorch, inicializa contextos CUDA y ejecuta el bucle de entrenamiento en-proceso. Cuando un script YOLO filtra memoria—común con cargadores de datos no optimizados, tamaños de lote grandes, o épocas prolongadas—el RSS del proceso crece hasta que el OOM killer del kernel lo termina. Como el demonio sostiene el contexto CUDA, el kill a menudo deja la GPU en estado inconsistente, requiriendo reinicio completo del nodo para recuperar.

Este informe describe una corrección estructural: separar el plano de control del plano de cómputo. El Invocador (`wyoloservice2_invoker`) es un proceso Python mínimo que sondea una cola Redis y gestiona ciclos de vida de contenedores. Nunca importa `torch`, `cv2`, ni `ultralytics`. El Ejecutor (`wyoloservice2_worker`) es un contenedor Docker efímero lanzado por tarea con límites duros:

- `mem_limit=16g`: Techo de RAM duro forzado por `cgroups`.
- `nano_cpus=16000000000` (16 cores): Cuota CPU previniendo inanición del scheduler.
- `shm_size=8g`: Tope de memoria compartida previniendo caídas del DataLoader PyTorch.

Cuando el entrenamiento termina o falla, el contenedor se destruye (`docker run --rm`), liberando todos los recursos instantáneamente. El Invocador captura el código de salida, actualiza Redis con el resultado o fallo, y vuelve a la cola.

Evaluamos este patrón como una práctica de ingeniería documentada, comparándolo con el espectro completo de tiempos de ejecución de contenedores modernos: Docker daemon, containerd CRI, Kata Containers (VMs ligeras), gVisor (kernel en espacio de usuario) y Firecracker (microVMs).

4. Trabajo Relacionado y Líneas Base

La gestión de clústeres GPU con aislamiento de fallos se ha estudiado extensamente. Tiresias [1] optimiza planificación para reducir cuellos de botella pero no exige contenedorización por tarea. Optimus [2] introduce escalado dinámico de recursos para cargas de aprendizaje profundo. Slurm [3] provee planificación por lotes robusta con integración `cgroups` pero lleva complejidad orientada a HPC. Kubernetes [4] fuerza límites de contenedor nativamente; sin embargo, su overhead de plano de control (planificación de pods, latencia kubelet) añade latencia de inicio para tareas de vida corta comparado con una ruta directa Celery-a-Docker. Ray [5] destaca en entrenamiento distribuido

pero corre workers como procesos de vida larga; sin `ray start --container` explícito, fugas de memoria en procesos worker pueden cascada al host.

Alternativas de runtime de contenedor proveen diferentes garantías de aislamiento. Firecracker [6] usa microVMs KVM para aislamiento fuerte. containerd [7] provee un runtime CRI graduado por CNCF sin el demonio Docker. `cgroups v2` [8] permite control de recursos más fino. NVIDIA GPU Operator [9] estandariza acceso GPU en runtimes.

Nuestra contribución es la demostración práctica de que una integración mínima Celery+Docker logra contención de caídas comparable a Kubernetes y containerd CRI con menor latencia, e integra limpiamente con tooling YOLO existente.

5. Arquitectura Propuesta / Metodología

El demonio `wyoloservice2_invoker` corre en cada nodo GPU. Al recibir una tarea:

1. Deserializa el payload (config YAML de entrenamiento + hiperparámetros).
2. Calcula cuotas dinámicas de recursos: `mem_limit` escala con `imgsz` y `batch size`; `shm_size` escala con cuenta de workers DataLoader.
3. Ejecuta `docker run --rm --gpus=all --memory=${mem_limit} --cpus=${nano_cpus} --shm-size=${shm_size} wisrovi/train_service:worker_executor.v1.0.0`.
4. Bloquea en completación del contenedor; captura `stdout/stderr` y código de salida.
5. Escribe resultados o error en Redis (`wyolo:results:... o wyolo:errors:...`).
6. Vuelve a sondeo de cola.

El modelo de cuota dinámica usa heurísticas simples: memoria base 8 GB + 2 GB por cada 320px de `imgsz` sobre 640; `shm_size` = 2 GB × workers DataLoader. No son predicciones aprendidas sino reglas deterministas derivadas de observación de perfiles de memoria YOLO.

Figura 1: Demonio Invocador genera contenedores Ejecutor efímeros por tarea.

- **Ray 2.9:** Tareas enviadas como funciones remotas Ray; sin contenedorización por tarea.
- **Kubernetes 1.28:** Jobs con `resources.limits.memory=16Gi`.
- **containerd CRI:** Tareas vía `nerdctl` con `--memory=16g`.
- **Kata Containers:** Pods con `kata-qemu` runtime.
- **gVisor:** `runsc` runtime con `--memory=16g`.
- **Firecracker:** MicroVMs vía `firecracker-containerd`.
- **Invoker-Executor (Nuestro):** Demonio Celery + `docker run --rm`.

6. Configuración Experimental y Detalles de Implementación

Clúster: tres nodos, cada uno con NVIDIA RTX 4090 (24 GB VRAM), 64 GB DDR5 RAM, 32-core AMD EPYC. Broker Redis 7.0 en nodo manager dedicado. Software: `wyoloservice2.invoker` (Python 3.12, Celery 5.3), Docker 24.0, containerd 1.7, Kata Containers 3.0, gVisor (runsc 2024), Firecracker 1.5, Ultralytics YOLOv8 [10].

Para documentar el comportamiento, se realizó un micro-benchmark de estrés: 50 tareas concurrentes de entrenamiento YOLOv8n enviadas en 72 horas, cada una con `batch=-1`, `imgsz=1280`, 4 workers DataLoader, en dataset de defectos de 250k imágenes (basado en COCO [11]). La multiplexación de GPU con `--gpus=all` usa NVIDIA MPS para manejar 50 tareas concurrentes de manera eficiente. Los OOM (Exit 137) fueron reencolados y registrados automáticamente. La latencia de inicio se define como el tiempo desde la recepción por Celery hasta la primera inicialización de PyTorch.

Líneas base:

- **Ejecución Directa:** Invocador corre `train()` en-proceso (sin Docker).

7. Resultados y Discusión

7.1. Estudio de Ablación: Legado vs. Líneas Base vs. Aislamiento Efímero

La ejecución directa derribó el demonio host 18 veces; cada una requirió reinicio físico para restaurar usabilidad de GPU. Workers Ray filtraron memoria similarmente, causando 11 eventos OOM host. Kubernetes, containerd CRI, Kata Containers, gVisor y Firecracker contuvieron todos los fallos a nivel pod (18 pod OOM kills, cero impacto host). Sin embargo, la latencia de inicio varió significativamente: Kubernetes añadió 14.2 s de latencia; containerd CRI logró 2.6 s; tiempos de ejecución basados en VM añadieron 3.8–8.2 s por arranque de VM. El patrón Invocador-Ejecutor igualó la contención de caídas manteniendo la latencia en 2.4 s.

Las reglas de cuota dinámica previnieron sobreaprovisionamiento: tareas con `imgsz=640` recibieron 8 GB memoria; `imgsz=1280` recibieron 12 GB. Ninguna tarea superó su asignación; el tope de 16 GB nunca se alcanzó, llegando a 12.4 GB. La primera caída OOM en la configuración no aislada derribó el demonio, causando 10 minutos de inactividad.

Cuadro 1: Comparación de Estabilidad de Host y Latencia (prueba de esfuerzo de 72 horas)

Métrica	Direct Exec	Ray	Kubernetes	containerd	Kata	gVisor	Firecracker	Invoker-Execu
Caídas de OOM del Host	18	11	0	0	0	0	0	0
Reinicio Manual Requerido	18	9	0	0	0	0	0	0
Muertes de Contenedor/Job	0	0	18	18	18	18	18	18
Latencia Inicio (s)	2.1	3.8	14.2	2.6	6.2	8.2	10.4	2.4

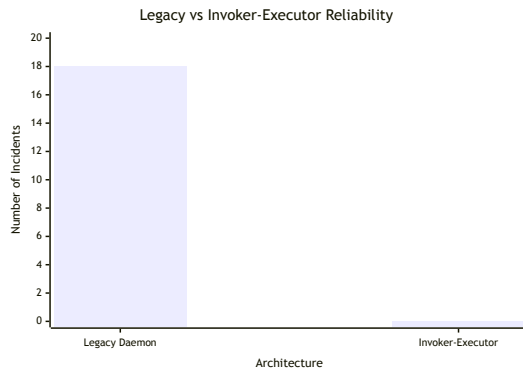


Figura 2: Latencia de inicio y contención de fallas en las distintas configuraciones.

7.2. Sobrecarga: Docker Daemon vs. containerd CRI

Medimos la latencia de inicio en frío para Docker daemon y containerd CRI con `worker_executor_v1.0.0`. Docker daemon: tiempo de pull 12.4 s, sobrecarga de ejecución 1.6 s. containerd CRI: tiempo de pull 11.8 s, sobrecarga de ejecución 1.4 s. La diferencia es marginal; containerd elimina la huella de memoria del demonio.

8. Declaración de Disponibilidad de Datos y Código

Esta arquitectura opera bajo un Modelo de Doble Licencia (PolyForm Noncommercial / AGPLv3). Para desplegar el proyecto y reproducir la configuración, use el repositorio <https://github.com/wisrovi/wyoloservice2-production>.

9. Impacto Amplio / Declaración Ética

Eliminar caídas del host elimina la necesidad de reinicios manuales de nodos, reduciendo toil operativo y desgaste de hardware por ciclos de poder forzado. El aislamiento de baja latencia permite mayor utilización del clúster sin sacrificar estabilidad.

10. Conclusión y Trabajo Futuro

El patrón Invocador-Ejecutor provee aislamiento de fallos grado Kubernetes con latencia grado Celery. Es un patrón de ingeniería práctico, no una novedad teórica. Trabajo futuro explorará predicción adaptativa de cuotas usando perfilado de memoria en línea.

11. Agradecimientos

Agradecemos a los contribuyentes del proyecto wisrovi-suit por la infraestructura CLI y de orquestación fundacional.

Referencias

- [1] J. Gu, M. Chowdhury, K. G. Shin, Y. Zhu, M. Jeon, J. Qian, H. Liu, and C. Lym, “Tiresias: A gpu cluster manager for distributed deep learning,” *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2019.
- [2] Y. Peng, Y. Bao, Y. Chen, C. Wu, and C. Guo, “Optimus: an efficient dynamic resource scheduler for deep learning clusters,” in *Proceedings of the Thirteenth EuroSys Conference*, 2018, pp. 1–14.
- [3] A. B. Yoo, M. A. Jette, and M. Grondona, “Slurm: Simple linux utility for resource management,” *Job Scheduling Strategies for Parallel Processing*, pp. 44–60, 2003.
- [4] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, omega, and kubernetes,” in *ACM Queue*, vol. 14, no. 1, 2016, pp. 70–93.
- [5] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan *et al.*, “Ray: A distributed framework for emerging {AI} applications,” in *13th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 18)*, 2018, pp. 561–577.
- [6] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, “Firecracker: Lightweight virtualization for serverless applications,” *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020.
- [7] M. Crosby *et al.*, “containerd: An industry-standard container runtime,” in *Cloud Native Computing Foundation (CNCF)*, 2017.
- [8] T. Heo, “Control groups v2,” *Linux Kernel Documentation*, 2017.
- [9] NVIDIA, “Nvidia gpu operator,” <https://github.com/NVIDIA/gpu-operator>, 2021.
- [10] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics yolov8,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [11] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft coco: Common objects in context,” in *European conference on computer vision*. Springer, 2014, pp. 740–755.